Last Time:

- [x] sequential decision-making
- [x] MDPs

This Time:

- [ ] Solving MDPs

lecture 3

HRI, FALL '26

Andrea Bajcsy

Recap & Important MDP Quantities

- **cumulative reward** (R): sum of (discounted) rewards
  - ↳ the utility of one rollout / state-action traj.

  For traj. / rollout $s_0, a_0, s_1, a_1, \ldots$

$$R(s_0, a_0, s_1, a_1, \ldots) = \sum_{t=0}^{\infty} \gamma^t r(s_t) \quad \text{or} \quad \sum_{t=0}^{\infty} \gamma^t r(s_t, a_t)$$

- **optimal policy** ($\pi^* : S \to A$): best action to take @ all states
  - ↳ maximizes expected discounted cumulative reward

$$\pi^* = \arg\max_{\pi} \mathbb{E}_{\tau \sim p_\pi(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right]$$

← expectation over all possible trajectories you generate via π

- **optimal value function** ($V^* : S \to \mathbb{R}$): expected sum of (discounted) rewards starting from s & acting optimally under π^*

$$V^*(s) := \mathbb{E}_{\tau \sim p_{\pi^*}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, \pi^*(s_t)) \,\middle|\, s_0 = s \right]$$

$$= \max_{\pi} \mathbb{E}_{\tau \sim p_\pi(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \,\middle|\, s_0 = s \right]$$

- **on-policy value function** ($V^\pi : S \to \mathbb{R}$): expected sum of (discounted) rewards starting s when acting under π

$$V^\pi(s) := \mathbb{E}_{\tau \sim p_\pi(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, \pi(s_t)) \,\middle|\, s_0 = s \right]$$

↑ discounted sum of rewards

↑ expectation over possible states you visit during rolling out π

Expand the expectation a bit more:

$$V^{\pi}(s) \triangleq \mathop{\mathbb{E}}_{\tau \sim p_{\pi}(\tau)}\left[\sum_{t=0}^{\infty} \gamma^t r(s_t, \pi(s_t)) \,\middle|\, s_0 = s\right]$$

← defn of value function

let's uncover a recursive structure

$$= \mathop{\mathbb{E}}_{\tau \sim p_{\pi}(\tau)}\left[\gamma^0 r(s_0, \pi(s_0)) + \sum_{t=1}^{\infty} \gamma^t r(s_t, \pi(s_t)) \,\middle|\, s_0 = s\right]$$

$$= \mathop{\mathbb{E}}_{\tau \sim p_{\pi}(\tau)}\left[\gamma^0 r(s_0, \pi(s_0)) + \gamma \sum_{t=1}^{\infty} \gamma^{t-1} r(s_t, \pi(s_t)) \,\middle|\, s_0 = s\right]$$

$$= \sum_{(s_0 = s, a_0, s_1, a_1, \ldots)} p_{\pi}(s_0, a_0, s_1, a_1, \ldots)\left[\gamma^0 r(s_0, \pi(s_0)) + \gamma \sum_{t=1}^{\infty} \gamma^{t-1} r(s_t, \pi(s_t)) \,\middle|\, s_0 = s\right]$$

Know from Markov: $\sum_{\tau} p(s_0, a_0, s_1, a_1, \ldots) = \sum_{s_0} p(s_0) \sum_{s_1} p(s_1 | s_0, a_0) \sum_{s_2} p(s_2 | s_1, a_1) \cdots$

Plug in & rearrange:

$$= \sum_{s_1} p(s_1 | s_0, \pi(s_0))\left[r(s_0, \pi(s_0)) + \gamma \mathop{\mathbb{E}}_{\tau \sim p_{\pi}(\tau)}\left[\sum_{t=1}^{\infty} \gamma^{t-1} r(s_t, \pi(s_t)) \,\middle|\, s_1\right] \,\middle|\, s_0\right]$$

distribution over trajectories starting from s_1!

$:= V^{\pi}(s_1)$ by defn

$$V^{\pi}(s) \triangleq \sum_{s' \in S} p(s' | s, \pi(s)) \cdot \left[r(s, \pi(s)) + \gamma V^{\pi}(s')\right]$$

↑ Bellman Equation ↑

We can also write / derive the BELLMAN OPTIMALITY EQN for V^*:

$$V^*(s) \triangleq \max_{a \in A} \sum_{s' \in S} p(s' | s, a) \cdot \left[r(s, a) + \gamma V^*(s')\right]$$

⍟ Bellman eqn. is so important b/c it breaks down a decision problem into smaller (one-step) problems! (also gives comp. savings over naive fwd. sim: exponential → polynomial in time horizon)

Q-value function:

↳ like a value function but you're already committed to taking a particular action, a.

$$Q^*(s,a) \triangleq \mathbb{E}_{\substack{s_1 \sim P(\cdot|s_0,a) \\ \tau \sim p_{\pi^*}(\tau), t>0}}\left[r(s_0,a) + \sum_{t=1}^{\infty} \gamma^t r(s_t, \pi^*(s_t)) \,\middle|\, s_0 = s, a_0 = a \right]$$

$s_1 \sim P(\cdot|s_0,a)$ ← take current action a now...

$\tau \sim p_{\pi^*}(\tau), t>0$ ← but afterwards act optimally!

$a_0 = a$ ↑ commit to initial action

There is a nice relationship btwn. V^* and Q^*:

$$V^*(s) = \max_{a \in A} Q^*(s,a)$$

$$Q^*(s,a) = \sum_{s' \in S} P(s'|s,a) \cdot \left[r(s,a) + \gamma V^*(s') \right]$$

$V^*(s') := \max_{a' \in A} Q^*(s',a')$

Intuition for Q-value: how "good" is taking action a?

Optimal policy: $\pi^*(s) = \arg\max_{a \in A} Q^*(s,a)$

Solving MDPs

Bellman equations are also useful b/c they help us solve MDPs!

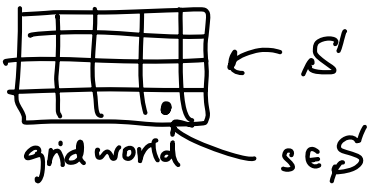
This comes from the recursive structure of Bellman eqn:

$$V(s) = \max_{a \in A} \sum_{s' \in S} P(s'|s,a)\left[r(s,a) + \gamma \cdot V(s') \right]$$

↑ we will use $V(s')$

$$V(s') = \max_{a \in A} \sum_{s'' \in S} P(s''|s',a)\left[r(s',a) + \gamma \cdot V(s'') \right]$$

VALUE ITERATION ALGORITHM:



$V_0[s] \leftarrow 0 \quad , \forall s \in S$

for $k = 0, 1, 2, \ldots$ until converged:

  for each state $s \in S$:

$$V_{k+1}[s] \leftarrow \max_a \sum_{s' \in S} P(s'|s,a)\left[r(s,a) + \gamma \cdot V_k[s']\right]$$

↑ another for loop over all $a \in A$

is another for loop over all next states

return converged $V[s]$

Q-VALUE ITERATION:

$Q_0[s,a] \leftarrow 0 \quad \forall s \in S, a \in A$ // store current + updated estimate of Q

for $k = 0, 1, 2, \ldots$ until converged:

  for each state $s \in S$ and action $a \in A$:

$$Q_{k+1}[s,a] \leftarrow \sum_{s' \in S} P(s'|s,a)\left[r[s,a] + \gamma \cdot \max_{a' \in A} Q_k[s', a']\right]$$

return converged $Q[s,a]$

Reinforcement learning (RL)

Q In MDPs, we assumed we know $P(s' \mid s, a)$ and rewards $r(s, a)$ .... but what if we don't?

In reality, it's hard to know P and r explicitly, so how do we obtain π^*?

A RL! It allows an agent to learn optimal policies by interacting with the environment.

⇒ agent tries out actions, observes rewards, and updates their policy to maximize rewards.

Model-based vs. Model-free RL

"models of the world" + rewards

↳ **High-level diff:** if the agent has access to (or learns) a transition function

(A) **Model-based:** 1) collect data from environment via some π

$$\mathcal{D} := \{(s_t, a_t, s_{t+1}, r_t)_{t=1}^{N}\}$$

2) use supervised learning to fit empirical MDP model:
   - ↳ count s' for each s, a; normalize; get $\hat{P}(s' \mid s, a)$
   - ↳ discover each $\hat{r}(s, a, s')$ when we experience (s, a, s')

3) solve MDP via techniques above (+more!)

(B) **Model-free:** 1) collect data from environment via some π

$$\mathcal{D} := \{(s_t, a_t, s_{t+1}, r_t)_{t=1}^{N}\}$$

2) Directly learn a Q-function or π via trial + error

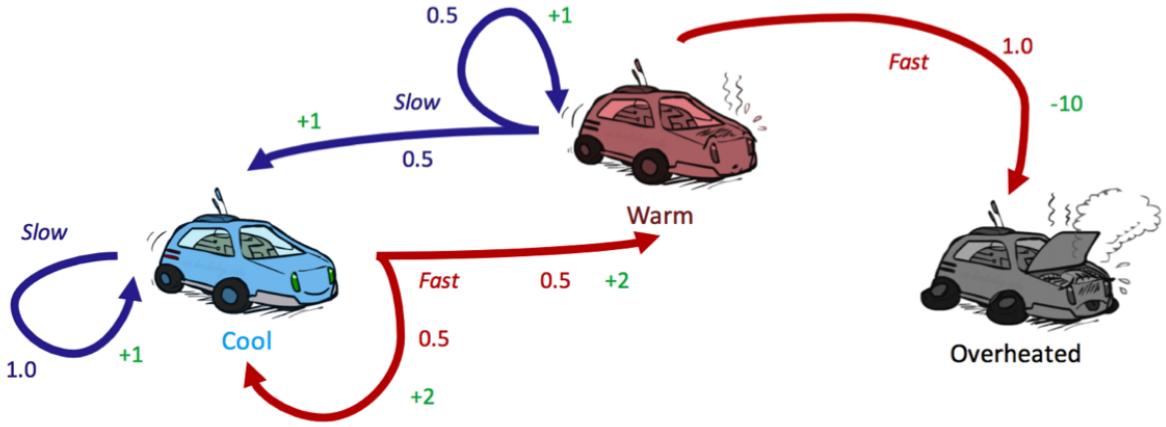
"Q-learning" "direct policy search"

⇒ for more depth on topic see:

- Sutton & Barto, "Reinforcement Learning."
- Russell & Norvig, "AI: A Modern Approach". Ch. 21

From CS188, UC Berkeley

Consider the motivating example of a racecar:



There are three possible states, $S = \{cool, warm, overheated\}$, and two possible actions $A = \{slow, fast\}$. Just like in a state-space graph, each of the three states is represented by a node, with edges representing actions. *Overheated* is a terminal state, since once a racecar agent arrives at this state, it can no longer perform any actions for further rewards (it's a *sink state* in the MDP and has no outgoing edges). Notably, for nondeterministic actions, there are multiple edges representing the same action from the same state with differing successor states. Each edge is annotated not only with the action it represents, but also a transition probability and corresponding reward. These are summarized below:

- **Transition Function**: $T(s,a,s')$
  - $T(cool, slow, cool) = 1$
  - $T(warm, slow, cool) = 0.5$
  - $T(warm, slow, warm) = 0.5$
  - $T(cool, fast, cool) = 0.5$
  - $T(cool, fast, warm) = 0.5$
  - $T(warm, fast, overheated) = 1$
- **Reward Function**: $R(s,a,s')$
  - $R(cool, slow, cool) = 1$
  - $R(warm, slow, cool) = 1$
  - $R(warm, slow, warm) = 1$
  - $R(cool, fast, cool) = 2$
  - $R(cool, fast, warm) = 2$
  - $R(warm, fast, overheated) = -10$

Let's see a few updates of value iteration in practice by revisiting our racecar MDP from earlier, introducing a discount factor of $\gamma = 0.5$:

	cool	**warm**	**overheated**
V_0	0	0	0
V_1			
V_2			

Recall that our ultimate goal in solving a MDP is to determine an optimal policy. This can be done once all optimal values for states are determined using a method called **policy extraction**. The intuition behind policy extraction is very simple: if you're in a state s, you should take the action a which yields the maximum expected utility. Not surprisingly, a is the action which takes us to the q-state with maximum q-value, allowing for a formal definition of the optimal policy:

$$\forall s \in S,\ \pi^*(s) = \underset{a}{\text{argmax}}\, Q^*(s,a) = \underset{a}{\text{argmax}} \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')]$$

It's useful to keep in mind for performance reasons that it's better for policy extraction to have the optimal q-values of states, in which case a single argmax operation is all that is required to determine the optimal action from a state.

	cool	**warm**
π_0		
π_1		
π_2		

Because terminal states have no outgoing actions, no policy can assign a value to one.